

Stop-and-Wait protocol simulation with acknowledge

- simulates the Stop-and-Wait protocol in the data link layer.
- sender machine sends only one data frame at a time. After sending a frame, the sender waits until it receives a valid acknowledgment from the receiver machine.
- if the acknowledgment is received correctly, the sender sends the next frame.
- if the data frame or the acknowledgment is lost or corrupted, the sender waits for a timeout and then retransmits the same frame.

```
import random
import time
```

Channel class

- simulates an unreliable communication channel.
- with this scenarios a frame can be:
 - lost
 - corrupted
 - delivered successfully
- if the frame is lost or corrupted, the method returns

```
class Channel:
```

```
    def __init__(self, loss_prob=0.2, corrupt_prob=0.2):
        self.loss_prob = loss_prob
        self.corrupt_prob = corrupt_prob

    def transmit(self, frame, label=""):

        if random.random() < self.loss_prob:
            print(f"    [Channel] {label} LOST in transit.")
            return None

        if random.random() < self.corrupt_prob:
            print(f"    [Channel] {label} CORRUPTED in transit.")
            return None

        return frame
```

Frame class

- represents both data frames and ACK frames.
- if `is_ack` is `False`, the frame is a data frame.
- if `is_ack` is `True`, the frame is an acknowledgment frame.

```
class Frame:
    def __init__(self, seq_num, data=None, is_ack=False):
        self.seq_num = seq_num
        self.data = data
        self.is_ack = is_ack

    def __repr__(self):
        frame_type = "ack" if self.is_ack else "DATA"
        return f"Frame({frame_type}, seq={self.seq_num}, data={self.data!r})"
```

Sender class

- represents the sender machine.
- sender keeps the sequence number of current frame.
- sequence number alternates between 0 and 1.
- sender machine sends a frame and waits for a valid ACK to back before sending the next frame.

```
class Sender:
    def __init__(self, timeout=1.0, max_retries=5):
        self.seq_num = 0
        self.timeout = timeout
        self.max_retries = max_retries

    def create_frame(self, data):
        return Frame(seq_num=self.seq_num, data=data, is_ack=False)

    def is_valid_ack(self, ack):
        return (
            ack is not None
            and ack.is_ack
            and ack.seq_num == self.seq_num
        )

    def move_to_next_sequence(self):
        self.seq_num = 1 - self.seq_num
```

Receiver class

- represents the receiver machine.
- receiver only accepts the frame that has the expected sequence number.
- if the frame is new, it is delivered to the network layer and an ACK is sent.
- if the frame is duplicate, it is discarded and the last ACK is sent again.
- if no frame arrives, no ACK is sent.

```
class Receiver:
    def __init__(self):
        self.expected_seq = 0
        self.last_ack = None

    def receive_frame(self, frame):
        if frame is None:
            print("[Receiver] Nothing received. No ack sent.")
            return None

        if frame.seq_num == self.expected_seq:
            print(f"[Receiver] New frame accepted: {frame}")
            print(f"    -> Delivered to network layer: {frame.data}")

            ack = Frame(seq_num=self.expected_seq, data=None, is_ack=True)
            self.last_ack = ack

            self.expected_seq = 1 - self.expected_seq

            return ack

        print(f"[Receiver] Duplicate frame with seq={frame.seq_num} discarded.")

        if self.last_ack is not None:
            print(f"[Receiver] Resending last ack: {self.last_ack}")

        return self.last_ack
```

Simulation methods

- considering the interactions with sender, receiver, and channel.
- for each packet we are dealing with:
 - sender creates a frame.
 - frame is sent through the channel which is unreliable.

- receiver processes the frame.
- receiver sends an ack if needed.
- ack also goes through the unreliable channel.
- ack is valid, sender moves to the next packet.
- if the ack is lost, corrupted, or invalid, timeout happens and the frame is retransmitted.

```
def sim_stop_and_wait(packets, loss_prob=0.3, corrupt_prob=0.2, timeout=0.3):

    channel = Channel(loss_prob=loss_prob, corrupt_prob=corrupt_prob)
    sender = Sender(timeout=timeout, max_retries=5)
    receiver = Receiver()

    for packet in packets:
        print(f"\n==== Sending packet: {packet!r} =====")

        frame = sender.create_frame(packet)
        attempts = 0
        success = False

        while attempts <= sender.max_retries and not success:
            attempts += 1

            print(f"[Sender] Sending {frame} (attempt {attempts})")

            arrived_frame = channel.transmit(
                frame,
                label=f"DATA seq={frame.seq_num}"
            )

            ack_from_receiver = receiver.receive_frame(arrived_frame)

            if ack_from_receiver is not None:
                arrived_ack = channel.transmit(
                    ack_from_receiver,
                    label=f"ACK seq={ack_from_receiver.seq_num}"
                )
            else:
                arrived_ack = None

            if sender.is_valid_ack(arrived_ack):
                print(f"[Sender] Valid ACK for seq={sender.seq_num} received.")
                print("[Sender] Moving to next packet.")

                sender.move_to_next_sequence()
                success = True
```

```

else:
    print("[Sender] Timeout occurred. Retransmitting...")
    time.sleep(sender.timeout)

if not success:
    print(f"[Sender] Failed to deliver packet {packet!r} after maximum retries.")

```

Initializes and log results

- notice:
 - `random.seed(42)` makes the random behavior repeatable.
 - you can remove it or change the number to get different results.

```

random.seed(42)

packets_to_send = [
    "Packet-1",
    "Packet-2",
    "Packet-3",
    "Packet-4"
]

sim_stop_and_wait(
    packets=packets_to_send,
    loss_prob=0.3,
    corrupt_prob=0.2,
    timeout=0.2
)

===== Sending packet: 'Packet-1' =====
[Sender] Sending Frame(DATA, seq=0, data='Packet-1') (attempt 1)
[Channel] DATA seq=0 CORRUPTED in transit.
[Receiver] Nothing received. No ack sent.
[Sender] Timeout occurred. Retransmitting...
[Sender] Sending Frame(DATA, seq=0, data='Packet-1') (attempt 2)
[Channel] DATA seq=0 LOST in transit.
[Receiver] Nothing received. No ack sent.
[Sender] Timeout occurred. Retransmitting...
[Sender] Sending Frame(DATA, seq=0, data='Packet-1') (attempt 3)
[Channel] DATA seq=0 LOST in transit.
[Receiver] Nothing received. No ack sent.
[Sender] Timeout occurred. Retransmitting...
[Sender] Sending Frame(DATA, seq=0, data='Packet-1') (attempt 4)
[Receiver] New frame accepted: Frame(DATA, seq=0, data='Packet-1')
-> Delivered to network layer: Packet-1

```

```

[Channel] ACK seq=0 CORRUPTED in transit.
[Sender] Timeout occurred. Retransmitting...
[Sender] Sending Frame(DATA, seq=0, data='Packet-1') (attempt 5)
[Channel] DATA seq=0 CORRUPTED in transit.
[Receiver] Nothing received. No ack sent.
[Sender] Timeout occurred. Retransmitting...
[Sender] Sending Frame(DATA, seq=0, data='Packet-1') (attempt 6)
[Channel] DATA seq=0 LOST in transit.
[Receiver] Nothing received. No ack sent.
[Sender] Timeout occurred. Retransmitting...
[Sender] Failed to deliver packet 'Packet-1' after maximum retries.

```

```

===== Sending packet: 'Packet-2' =====
[Sender] Sending Frame(DATA, seq=0, data='Packet-2') (attempt 1)
[Channel] DATA seq=0 CORRUPTED in transit.
[Receiver] Nothing received. No ack sent.
[Sender] Timeout occurred. Retransmitting...
[Sender] Sending Frame(DATA, seq=0, data='Packet-2') (attempt 2)
[Channel] DATA seq=0 LOST in transit.
[Receiver] Nothing received. No ack sent.
[Sender] Timeout occurred. Retransmitting...
[Sender] Sending Frame(DATA, seq=0, data='Packet-2') (attempt 3)
[Receiver] Duplicate frame with seq=0 discarded.
[Receiver] Resending last ack: Frame(ack, seq=0, data=None)
[Channel] ACK seq=0 LOST in transit.
[Sender] Timeout occurred. Retransmitting...
[Sender] Sending Frame(DATA, seq=0, data='Packet-2') (attempt 4)
[Receiver] Duplicate frame with seq=0 discarded.
[Receiver] Resending last ack: Frame(ack, seq=0, data=None)
[Channel] ACK seq=0 LOST in transit.
[Sender] Timeout occurred. Retransmitting...
[Sender] Sending Frame(DATA, seq=0, data='Packet-2') (attempt 5)
[Receiver] Duplicate frame with seq=0 discarded.
[Receiver] Resending last ack: Frame(ack, seq=0, data=None)
[Channel] ACK seq=0 CORRUPTED in transit.
[Sender] Timeout occurred. Retransmitting...
[Sender] Sending Frame(DATA, seq=0, data='Packet-2') (attempt 6)
[Receiver] Duplicate frame with seq=0 discarded.
[Receiver] Resending last ack: Frame(ack, seq=0, data=None)
[Channel] ACK seq=0 LOST in transit.
[Sender] Timeout occurred. Retransmitting...
[Sender] Failed to deliver packet 'Packet-2' after maximum retries.

```

```

===== Sending packet: 'Packet-3' =====
[Sender] Sending Frame(DATA, seq=0, data='Packet-3') (attempt 1)
[Channel] DATA seq=0 LOST in transit.

```

```
[Receiver] Nothing received. No ack sent.
[Sender] Timeout occurred. Retransmitting...
[Sender] Sending Frame(DATA, seq=0, data='Packet-3') (attempt 2)
[Receiver] Duplicate frame with seq=0 discarded.
[Receiver] Resending last ack: Frame(ack, seq=0, data=None)
[Sender] Valid ACK for seq=0 received.
[Sender] Moving to next packet.

==== Sending packet: 'Packet-4' ====
[Sender] Sending Frame(DATA, seq=1, data='Packet-4') (attempt 1)
[Receiver] New frame accepted: Frame(DATA, seq=1, data='Packet-4')
    -> Delivered to network layer: Packet-4
[Sender] Valid ACK for seq=1 received.
[Sender] Moving to next packet.
```